

4. 모듈

모듈(module)은 코드를 관리하는 가장 작은 단위로 코드 관리와 재활용이 편해 집니다. ES6 부터 import, export 문을 추가 되므로 전통적인 방식의 소스 복잡도를 개선 할 수 있습니다.

모듈을 사용하기 위해서는 자바스크립트 파일(.js)를 기능 단위로 다음과 같은 규칙으로 코드를 작성합니다. 모듈의 파일 확장자는 mjs를 권장합니다.

script 태그에 type="module" 어트리뷰트를 추가하면 로드된 자바스크립트 파일은 모듈로서 동작합니다.

1. 내보내기

모듈을 작성하고 여러 값을 공유하는 *****이름으로 내보내기*****와 기본값 하나만 공유하는 **"기본으로 내보내기"** 방식이 있습니다.

1-1. 이름으로 내보내기

소스 파일에 작성된 식별자(변수, 상수, 함수, 클래스등)를 공유하는 방식으로 export 지시자를 사용해서 중괄호({}) 안에 공유하고자 하는 식별자를 쉼표로 나열 합니다.

즉, 객체 지향 프로그램에서 이야기 하는 공개, 비공개를 만들어 특정 기능 또는 정보를 은닉할 수 있습니다.

```
// 외부에 공개할 변수 및 함수
const helloMessage = "안녕하세요";
const fnHello = (name, message) => {
  return fnHelloMsg(name, message);
}

// 외부에 공개하지 않는 함수
const fnHelloMsg = (name, message) => {
  return `${name}님 ${message}`;
}

// 외부에 공개할 변수 및 함수 선언
export { helloMessage, fnHello } ;
```

1-2. 기본으로 내보내기

소스 파일에 있는 작성된 식별자 중 하나를 대표로 지정해서 공유하는 방식으로 export 지시자 다음에 default 지시자를 추가 하면 됩니다. 이때 모듈 하나당 하나의 함수나 클래스만 공유할 수 있고 var, let, const는 사용할 수 없습니다.

기본 식별자와 다른 것을 외부로 공개 하기 위해서는 export 지시자로 선언 할 수 있습니다.

{% code title="./default.js" lineNumbers="true" %}

```
const helloMessage = "반갑습니다. 안녕하세요.";
```

```
// 기본 식별자로 가져 오기 위한 함수
const fnOpenHelloMsg = name => {
  return fnHelloMsg(name);
}

const fnHelloMsg = name => {
  return `${name}님 ${helloMessage}`;
}

export { fnHelloMsg } ;
// 기본 식별자 외부 공개
export default fnOpenHelloMsg ;
```

{% endcode %}

2. 가져오기

import 지시자를 통해서 외부 모듈의 기능을 사용할 수 있습니다.

import 지시자를 통해서 가져오는 방법은 export로 내보낸 식별자를 선택적으로 지정 하여 가져 오는 방법, 모든 식별자 가져 오는 방법, 기본식별자 가져오는 방법이 있습니다.

```
// 선택적으로 식별자 가져 오기
import {helloMessage, fnHello} from './named.js';

// 지정한 파일의 모든 요소 가져 오기
// as로 모듈을 담을 객체명을 지정 해야 한다,
// 사용 : myNamed.변수, myNamed.함수
import * as myNamed from './named.js';

// 기본식별자로 지정된 함수는 {}를 사용할 필요가 없습니다.
import fnOpenHelloMsg from './default.js';

// 기본 식별자와 다른 공개된 식별자 혼용으로 가져오기
import fnOpenHelloMsg, {fnHelloMsg} from './default.js';
```

3. 모듈 예제

모듈을 사용하기 위해서는 <script type="module">같은 속성을 설정해 해당 스크립트가 모듈이란 걸 브라우저가 알 수 있게 해줘야 합니다.

{% code title="index.html" lineNumbers="true" %}

```
<!DOCTYPE html>
<html lang="kr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>HTML</title>
</head>
<body>
  선택 식별자 : named/fnHello :: <Label id="helloMsg" ></Label></br>
  기본 식별자 : default/fnOpenHelloMsg :: <Label id="helloMsgDefault" >
</Label></br>
  선택 식별자 : default/fnHelloMsg :: <Label id="helloMsgDefaultExport" >
</Label></br>
</body>
<script type="module">

  import {helloMessage, fnHello} from './named.js';
  import fnOpenHelloMsg, {fnHelloMsg} from './default.js';

  document.querySelector("#helloMsg")
    .innerHTML = fnHello('홍길동', helloMessage);

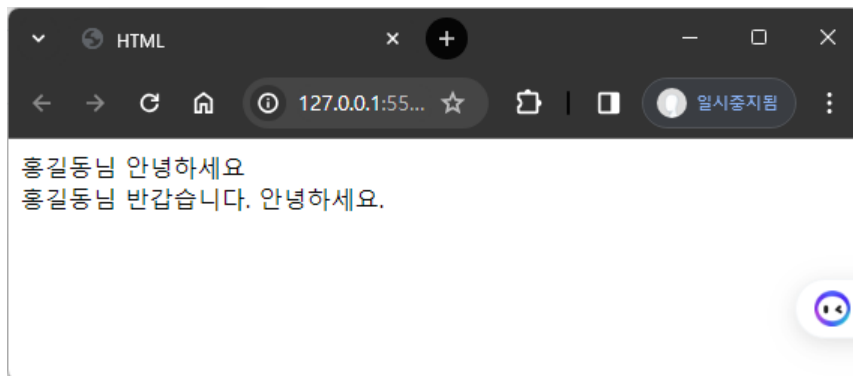
  document.querySelector("#helloMsgDefault")
    .innerHTML = fnOpenHelloMsg('홍길동');

  document.querySelector("#helloMsgDefaultExport")
    .innerHTML = fnHelloMsg('홍길동');

</script>
</html>

```

{% endcode %}



모듈 가져오기 수행 된 모습

- 14 라인 : 스크립트가 모듈이라는 것을 웹 브라우저에게 알림
- 15 라인 : named.js 모듈 중 helloMessage, fnHello 사용 선언
- 16 라인 : default.js 모듈의 기본 식별자와 fnHelloMsg 사용 선언
 - 기본 식별자는 중괄호를 사용하지 않아도 된다.
- 19라인, 22 라인, 25라인 : 사용된 선언된 모듈 사용

모듈의 모든 요소를 사용하기 위해서는 * 를 사용 하고 모듈을 담고 있을 객체를 as로 지정 한 후 사용해야 합니다.

```

<script type="module">
  // 지정한 파일의 모든 요소 가져 오기

```

```
import * as named from './named.js';

document.querySelector("#helloMsg")
    .innerHTML = named.fnHello('홍길동'
        , named.helloMessage);

</script>
```

4. 함수 선언식, 표현식둘모 선언 예시

export 지시자는 변수나 함수등을 만들고 별도 선언을 하여 작성해도 되고 변수나 함수를 만들때 선언 하여도 됩니다.

```
export const helloMessage = "안녕하세요";

export const fnHello = (name, message) => {
    return fnHelloMsg(name, message);
}

export default function fnOpenHelloMsg (name) {
    return fnHelloMsg(name);
}

export function fnHelloMsg (name) {
    return `${name}님 ${helloMessage}`;
}
```

5. 모듈의 특성

- 모듈은 항상 엄격 모드(use strict)로 실행됩니다.
- 모듈은 자신만의 스코프가 있습니다. 따라서 모듈 내부에서 정의한 변수나 함수는 다른 스크립트에서 접근할 수 없습니다.
- 모듈은 단 한 번만 실행되고 실행된 모듈은 필요한 곳에 공유되므로 어느 한 모듈에서 해당객체를 수정하면 다른 모듈에서도 변경사항을 확인할 수 있습니다. 주의해서 사용을 해야 합니다.
- 모듈 스크립트(<script type="module">)는 지연 실행되기 때문에 문서 전체가 처리되기 전까지 실행되지 않습니다. html 파일에서 일반 자바스크립트(<script>) 는 동시에 병렬로 실행 되므로 자바스크립트가 로드되기 전에 실행이 되면 "undefined"오류가 발생 합니다.
 - <script async type="module">로 선언이 되어 있으면 일반 자바 스크립트 처럼 바로 실행이 됩니다.

5-1. 주의사항

다음 코드를 통해 모듈은 단 한 번만 실행되고 실행된 모듈은 필요한 곳에 공유되는 것을 확인할 수 있습니다.

```
{% code title="data.js" lineNumbers="true" %}
```

```
export let user = {
    name: '홍길동'
};
```

```
{% endcode %}
```

```
{% code title="named.js" lineNumbers="true" %}
```

```
import {user} from './data.js';

export const helloMessage = "안녕하세요";
export const fnHello = (message) => {
  return `${user.name}님 ${message}`;
}
```

```
{% endcode %}
```

```
{% code title="default.js" lineNumbers="true" %}
```

```
import {user} from './data.js';

const helloMessage = "안녕하세요";
user.name = "김길동"

export function fnHelloMsg (name) {
  return `${user.name}님 ${helloMessage}`;
}
```

```
{% endcode %}
```

```
{% code title="index.html" lineNumbers="true" %}
```

```
<!DOCTYPE html>
<html lang="kr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML</title>
</head>
<body>
  선택 식별자 : named/fnHello :: <Label id="helloMsg" ></Label></br>
  선택 식별자 : default/fnHelloMsg :: <Label id="helloMsgDefaultExport" >
</Label></br>
</body>
<script type="module">

  import {helloMessage, fnHello} from './named.js';
  import {fnHelloMsg} from './default.js';

  document.querySelector("#helloMsg")
    .innerHTML = fnHello('홍길동', helloMessage);
```

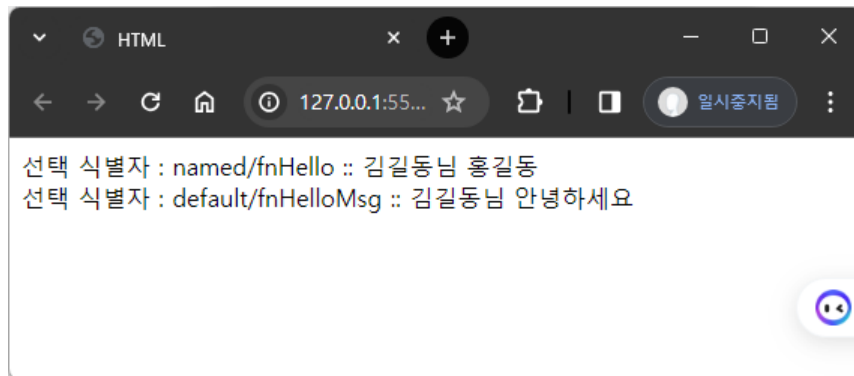
```

document.querySelector("#helloMsgDefaultExport")
    .innerHTML = fnHelloMsg('홍길동');

</script>
</html>

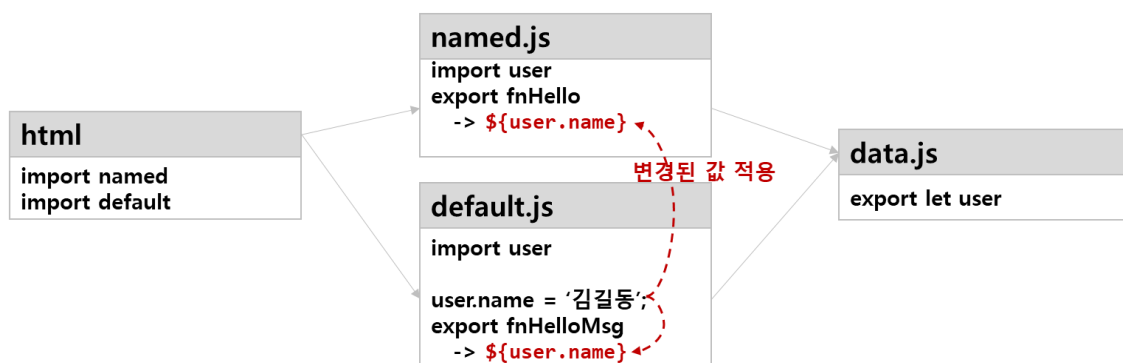
```

{% endcode %}



수행 결과

default.js에서 변경된 값이 name.js에 user.name에 적용되는 것을 확인 할 수 있습니다.



모듈에 대한 자세한 사항은 <https://ko.javascript.info/modules-intro> 참고 하시면 됩니다.